

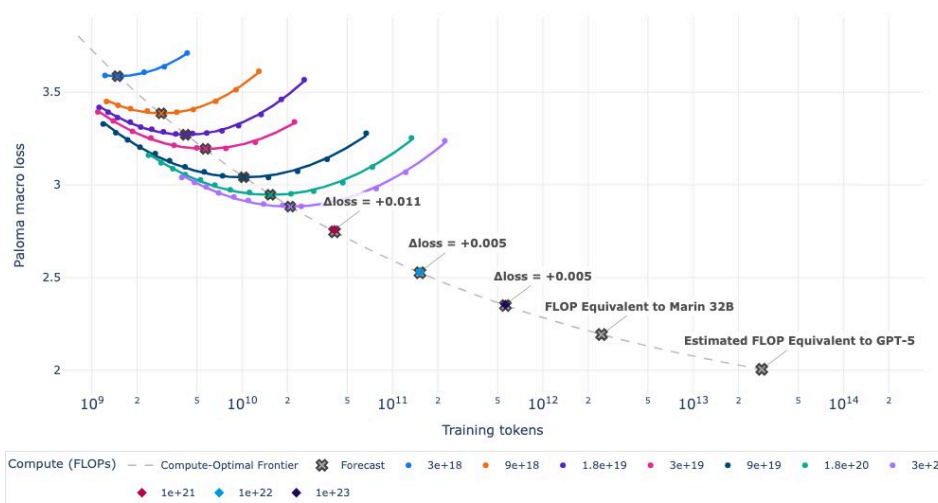
lecture_02.py



```

1 import torch.nn.functional as F
2 import timeit
3 import torch
4 from typing import Iterable
5 from torch import nn
6 from torch.utils.checkpoint import checkpoint
7 import numpy as np
8 from edtrace import text, image, link
9 from lecture_util import article_link
10 from einops import rearrange, einsum, reduce
11 from references import deepseek_v3_2_2025, adagrad_2011, nemotron_3_2025
12 from gpu_util import cuda_if_available
13 from facts import h100_flop_per_sec, h100_bytes_per_sec
14
15
16 def main():
17     Announcements:
18     • Join the CS336 slack
19     • Sign up on Modal with your Stanford email
20     • Read the AI policy guide
21     • Read the cluster guide
22
23     Marin 1e23 FLOPs run finished and matched forecasts
24 
```

Delphi Scaling Suite Results



```

25
26 Last lecture: overview, tokenization
27 Today: resource accounting (systems)
28
29 Recall: what's the best model one can train given fixed resources (compute, memory)?
30 In other words: maximize (computational) efficiency.
31 Precursor: We need to understand the resources (compute, memory) for a given computation?
32
33 motivating_questions()
34
35 What knowledge to take away:
36 • Mechanics: straightforward (PyTorch semantics)
37 • Mindset: resource accounting (remember to do it)
38 • Intuitions: get a sense of how resources are spent, no ML magic today
39
40 # Memory accounting
41 tensors_basics()
42 tensors_memory()
43 tensors_on_gpus()

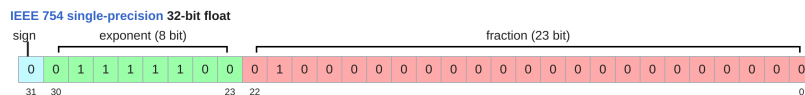
```

```

44
45 # Compute accounting
46 tensor_einops()
47 tensor_operations_flops()
48
49 arithmetic_intensity()
50
51 # Full example
52 deep_linear_network()
53 gradients_basics()
54 gradients_flops()
55 optimizer()
56 train_loop()
57
58 # More memory optimizations
59 gradient_accumulation()
60 activation_checkpointing()
61
62 Summary:
63 • Everything is operations on tensors (parameters, gradients, activations, optimizer states, data)
64 • einops: better way to think about tensor operations
65 • 6 (# data points) (# parameters) FLOPs per training step
66 • Arithmetic intensity / roofline analysis: compute-bound or memory-bound?
67 • Matrix multiplications are compute-bound, elementwise operations are memory-bound
68 • Gradient accumulation, activation checkpointing: reduce memory to use bigger batch sizes
69
70
71 def motivating_questions():
72     Question: How long would it take to train a 70B parameter model on 15T tokens on 1024 B100s?
73     total_flops = 6 * 70e9 * 15e12
74     h100_flop_per_sec = 1979e12 / 2
75     mfu = 0.5
76     flops_per_day = h100_flop_per_sec * mfu * 1024 * 60 * 60 * 24
77     days = total_flops / flops_per_day
78
79     Question: What's the largest model that can you can train on 8 H100s using AdamW (naively)?
80     h100_bytes = 80e9
81     bytes_per_parameter = 2 + 2 + (4 + 4) # parameters, gradients, optimizer state
82     num_parameters = (h100_bytes * 8) / bytes_per_parameter
83     Caveat: activations are not accounted for (depends on batch size and sequence length).
84
85     This is a rough back-of-the-envelope calculation.
86
87
88 def tensors_basics():
89     Tensors are the basic building block for storing everything:
90     • parameters, gradients, optimizer state, data, activations.
91
92     Example: parameters of the DeepSeek v3.2 model [DeepSeek-AI+ 2025]
93     [DeepSeek v3.2 model on Hugging Face]
94
95     x = torch.zeros(4) # vector
96     x = torch.zeros(4, 8) # matrix
97     x = torch.zeros(4, 8) # rank-3 tensor
98
99
100 def tensors_memory():
101     Almost everything (parameters, gradients, activations, optimizer states) are stored as floating point numbers.
102
103     float32
104     [Wikipedia]

```

105



106

The float32 data type (also known as fp32 or single precision) is the default.

107

Traditionally, in scientific computing, float32 is the baseline; you could use double precision (float64) in some cases.

108

In deep learning, you can be a lot sloppier.

109

110

Let's examine memory usage of these tensors.

111

Memory is determined by the (i) number of values and (ii) data type of each value.

112

```
x = torch.zeros(4, 8)
```

113

```
assert x.dtype == torch.float32 # Default type
```

114

```
assert x.numel() == 4 * 8
```

115

```
assert x.element_size() == 4 # Float is 4 bytes
```

116

```
assert get_memory_usage(x) == 4 * 8 * 4 # 128 bytes
```

117

118

One matrix in the feedforward layer of GPT-3:

119

```
assert get_memory_usage(torch.empty(12288 * 4, 12288)) == 2304 * 1024 * 1024 # 2.3 GB
```

120

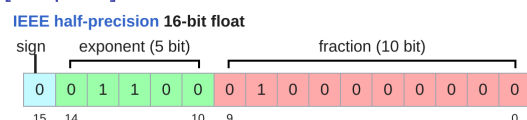
121

float16

122

[Wikipedia]

123



124

The float16 data type (also known as fp16 or half precision) cuts down the memory.

125

```
x = torch.zeros(4, 8, dtype=torch.float16)
```

126

```
assert x.element_size() == 2
```

127

However, the dynamic range (especially for small numbers) isn't great.

128

```
x = torch.tensor([1e-8], dtype=torch.float16)
```

129

```
assert x == 0 # Underflow!
```

130

If this happens when you train, you can get instability.

131

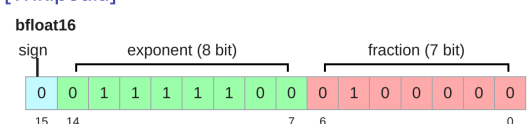
132

bfloat16

133

[Wikipedia]

134



135

Google Brain developed bfloat (brain floating point) in 2018 to address this issue.

136

bfloat16 uses the same memory as float16 but has the same dynamic range as float32!

137

The only catch is that the resolution is worse, but this matters less for deep learning.

138

```
x = torch.tensor([1e-8], dtype=torch.bfloat16)
```

139

```
assert x != 0 # No underflow!
```

140

141

Implications on training:

142

- Training with float32 works, but requires lots of memory.

143

- Training with fp8, float16 and even bfloat16 is risky, and you can get instability.

144

145

Mixed precision training [Micikevicius+ 2017]

146

- Use bf16 for parameters, activations, and gradients

147

- Use fp32 for optimizer states

148

149

Pytorch has an automatic mixed precision (AMP) library. [docs]

150

Tries to cast things into bf16 when safe (matmuls, not exp).

151

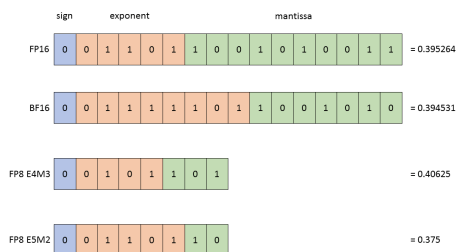
152

fp8

153

In 2022, fp8 was standardized, motivated by machine learning workloads [primer](#).

154



155

H100s support two variants of FP8: E4M3 (range [-448, 448]) and E5M2 ([-57344, 57344]).

156

Reference: [\[Micikevicius+ 2022\]](#)

157

158

fp4

159

In 2025, NVIDIA developed [nvfp4](#)

160

Only 4 bits per value!

161

Values: -6, -4, -3, -2, -1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5, 2, 3, 4, 6

162

Use a separate scale factor per block

163

Nemotron 3 Super was trained in NVFP4 [\[NVIDIA+ 2025\]](#)

164

165

Some of this is done in NVIDIA libraries outside of user control.

166

167

```
def tensors_on_gpus():
```

169

By default, tensors are stored in CPU memory.

170

```
x = torch.zeros(32, 32)
```

171

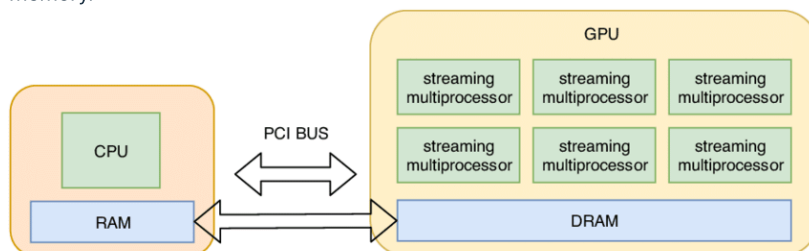
```
assert x.device == torch.device("cpu")
```

172

173

However, in order to take advantage of the massive parallelism of GPUs, we need to move them to GPU memory.

174



175

176

Let's first see if we have any GPUs.

177

```
if not torch.cuda.is_available():
```

178

```
    return
```

179

180

```
num_gpus = torch.cuda.device_count()
```

181

```
for i in range(num_gpus):
```

182

```
    properties = torch.cuda.get_device_properties(i)
```

183

184

```
memory_allocated = torch.cuda.memory_allocated()
```

185

186

```
text("Move the tensor to GPU memory (device 0).")
```

187

```
y = x.to("cuda:0")
```

188

```
assert y.device == torch.device("cuda", 0)
```

189

190

```
text("Or create a tensor directly on the GPU:")
```

191

```
z = torch.zeros(32, 32, device="cuda:0")
```

192

193

```
new_memory_allocated = torch.cuda.memory_allocated()
```

194

```
memory_used = new_memory_allocated - memory_allocated
```

195

```
assert memory_used == 2 * (32 * 32 * 4) # 2 32x32 matrices of 4-byte floats
```

196

197

198

199

```
def tensor_einops():
```

200

```
    einops_motivation()
```

```

201
202 Einops is a library for manipulating tensors where dimensions are named.
203 It is inspired by Einstein summation notation (Einstein, 1916).
204 \[Einops tutorial\]
205
206 einops_einsum()
207 einops_reduce()
208 einops_rearrange()
209
210
211 def einops_motivation():
212     Easy to mess up the dimensions (what is -2, -1?)...
213     Traditional PyTorch code:
214     x = torch.ones(2, 2, 3)      # batch seq hidden
215     y = torch.ones(2, 2, 3)      # batch seq hidden
216     z = x @ y.transpose(-2, -1)  # batch seq seq
217     Easy to mess up the dimensions (what is -2, -1?)...
218
219
220 def einops_einsum():
221     Einsum is generalized matrix multiplication with good bookkeeping.
222
223     x = torch.ones(3, 4) # seq1 hidden
224     y = torch.ones(4, 3) # hidden seq2
225
226     # Old way
227     z = x @ y # seq1 seq2
228
229     # New (einops) way
230     z = einsum(x, y, "seq1 hidden, hidden seq2 -> seq1 seq2")
231
232     Let's try a more complex example...
233
234     x = torch.ones(2, 3, 4) # batch seq1 hidden
235     y = torch.ones(2, 3, 4) # batch seq2 hidden
236
237     # Old way
238     z = x @ y.transpose(-2, -1) # batch seq1 seq2
239
240     # New (einops) way
241     z = einsum(x, y, "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
242     Dimensions that are not named in the output are summed over.
243
244     # Or can use `...` to represent broadcasting over any number of dimensions
245     z = einsum(x, y, "... seq1 hidden, ... seq2 hidden -> ... seq1 seq2")
246
247
248 def einops_reduce():
249     You can reduce a single tensor via some operation (e.g., sum, mean, max, min).
250     x = torch.ones(2, 3, 4) # batch seq hidden
251
252     # Old way
253     y = x.sum(dim=-1)
254
255     # New (einops) way
256     y = reduce(x, "... hidden -> ...", "sum")
257
258
259 def einops_rearrange():
260     Sometimes, a dimension represents two dimensions
261     ...and you want to operate on one of them.
262
263     x = torch.ones(3, 8) # seq total_hidden
264     ...where total_hidden is a flattened representation of heads * hidden1

```

```

265 w = torch.ones(4, 4) # hidden1 hidden2
266
267 # Break up `total_hidden` into two dimensions (`heads` and `hidden1`
268 x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)
269
270 # Perform the transformation by `w`
271 x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
272
273 # Combine `heads` and `hidden2` back together
274 x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")
275
276
277 def tensor_operations_flops():
278     Having gone through all the operations, let us examine their computational cost.
279
280     A floating-point operation (FLOP) is a basic operation like addition (x + y) or multiplication (x y).
281
282     Two terribly confusing acronyms (pronounced the same!):
283     • FLOPs: floating-point operations (measure of computation done)
284     • FLOPs/s: floating-point operations per second (also written as FLOPS), which is used to measure the speed
      of hardware.

```

Intuitions

```

287 Training GPT-3 (2020) took 3.14e23 FLOPs. \[article\]
288 Training GPT-4 (2023) is speculated to take 2e25 FLOPs. \[article\]
289
290 H100 has a peak performance of 1979 teraFLOP/s with sparsity, 50% without \[spec\]
291 h100_flop_per_sec = 1979e12 / 2
292
293 8 H100s for 2 weeks:
294 total_flops = 8 * (60 * 60 * 24 * 7) * h100_flop_per_sec
295
296

```

Linear model

```

297 As motivation, suppose you have a linear model.
298 • We have n points
299 • Each point is d-dimensional
300 • The linear model maps each d-dimensional vector to a k outputs
301

```

```

302 if torch.cuda.is_available():
303     B = 16384 # Number of points
304     D = 32768 # Dimension
305     K = 8192 # Number of outputs
306 else:
307     B = 1024
308     D = 256
309     K = 64
310
311 x = torch.ones(B, D, device=cuda_if_available())
312 w = torch.randn(D, K, device=cuda_if_available())
313 y = x @ w

```

```

314 We have one multiplication (x[i][j] * w[j][k]) and one addition per (i, j, k) triple.
315 actual_num_flops = 2 * B * D * K
316
317

```

FLOPs of other operations

```

318 • Elementwise operation on a m x n matrix requires O(m n) FLOPs.
319 • Addition of two m x n matrices requires m n FLOPs.
320 In general, no other operation that you'd encounter in deep learning is as expensive as matrix multiplication for
    large enough matrices.
321
322 Interpretation:
323 • B is the number of data points
324 • (D K) is the number of parameters
325 • FLOPs for forward pass is 2 (# tokens) (# parameters)

```

It turns out this generalizes to Transformers (to a first-order approximation).

How do our FLOPs calculations translate to wall-clock time (seconds)?

Let us time it!

```
actual_time = time_matmul(x, w)
```

```
actual_flop_per_sec = actual_num_flops / actual_time
```

Each GPU has a specification sheet that reports the peak performance.

- Example: [\[H100 spec\]](#)

Note that the FLOP/s depends heavily on the data type!

```
promised_flop_per_sec = get_promised_flop_per_sec(x.dtype)
```

Model FLOPs utilization (MFU)

Definition: (actual FLOP/s) / (promised FLOP/s) [ignore communication/overhead]

```
mfu = actual_flop_per_sec / promised_flop_per_sec
```

Usually, MFU of ≥ 0.5 is quite good (and will be higher if matmuls dominate)

Summary

- Matrix multiplications dominate: (2 m n p) FLOPs
- FLOP/s depends on hardware (B200 >> H100) and data type (bfloat16 >> float32)
- Model FLOPs utilization (MFU): (actual FLOP/s) / (promised FLOP/s)

```
def gradients_basics():
```

So far, we've constructed tensors (which correspond to either parameters or data) and passed them through operations (forward).

Now, we're going to compute the gradient (backward).

As a simple example, let's consider the simple linear model:

$$y = 0.5 (x * w - 5)^2$$

Forward pass: compute loss

```
x = torch.tensor([1., 2, 3])
```

```
w = torch.tensor([1., 1, 1], requires_grad=True) # Want gradient
```

```
pred_y = x @ w
```

```
loss = 0.5 * (pred_y - 5).pow(2)
```

Backward pass: compute gradients

```
loss.backward()
```

```
assert loss.grad is None
```

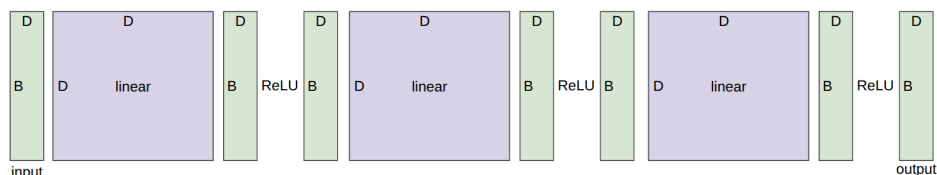
```
assert pred_y.grad is None
```

```
assert x.grad is None
```

```
assert torch.equal(w.grad, torch.tensor([1, 2, 3]))
```

```
def gradients_flops():
```

Let us count the FLOPs for computing gradients.



```
B = 1024 # Number of points
```

```
D = 256 # Dimension
```

Define a simplified model (2-layer linear network):

```
x = torch.ones(B, D, device=cuda_if_available())
```

```
w1 = torch.randn(D, D, device=cuda_if_available(), requires_grad=True)
```

```
w2 = torch.randn(D, D, device=cuda_if_available(), requires_grad=True)
```

```

383
384 # Forward pass
385 h1 = einsum(x, w1, "batch in, in out -> batch out") # x
386 h2 = einsum(h1, w2, "batch in, in out -> batch out") # h1
387 loss = (h2.mean() - 0)**2 # Regress everything to 0 (arbitrary)
388
389 # Backward pass
390 h1.retain_grad() # For debugging
391 h2.retain_grad() # For debugging
392 loss.backward()
393
394

```

Zoom in on one layer

Let's focus on the second layer $h1 \rightarrow w2 \rightarrow h2$.

Forward pass: Recall the number of forward FLOPs (bf16):

```
num_forward_flops = 2 * B * D * D
```

Backward pass: How many FLOPs is running the backward pass?

We need to compute:

- $h1.grad = d \text{ loss} / d h1$
- $w2.grad = d \text{ loss} / d w2$

```
h1_grad = einsum(h2.grad, w2, "batch out, in out -> batch in")
```

```
assert torch.allclose(h1.grad, h1_grad)
```

```
w2_grad = einsum(h2.grad, h1, "batch out, batch in -> in out")
```

```
assert torch.allclose(w2.grad, w2_grad)
```

```
num_backward_flops = (2 * B * D * D) + (2 * B * D * D)
```

Note that the backward pass is 2x more expensive than the forward pass.

Consider all layers

This was just for $w2$, need to apply it to all parameters in the network.

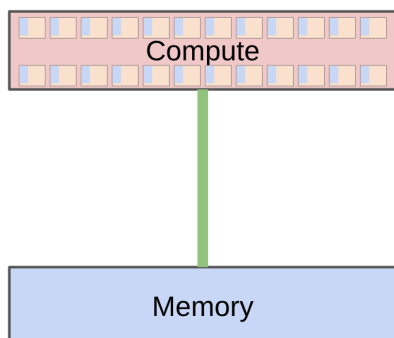
Putting it together:

- Forward pass: 2 (# data points) (# parameters) FLOPs
- Backward pass: 4 (# data points) (# parameters) FLOPs
- Total: 6 (# data points) (# parameters) FLOPs

This is for multilayer perceptrons (MLPs)

...but it turns out to be a good approximation for Transformers for short context lengths as well.

```
def arithmetic_intensity():
```



How to compute a thing:

1. Send inputs from memory to accelerator
2. Perform computation
3. Send outputs from accelerator to memory

How long does this take?


```

436
437 Depends on two things:
438 1. Accelerator speed (FLOP/s)
439 1. Memory bandwidth (bytes/s)
440 assert h100_flop_per_sec == 1979e12 / 2 # Half without sparsity
441 assert h100_bytes_per_sec == 3.35e12
442
443 arithmetic_intensity_relu()
444 arithmetic_intensity_gelu()
445 arithmetic_intensity_dot_product()
446 arithmetic_intensity_matrix_vector_product()
447 arithmetic_intensity_matmul()
448
449 # Let's visualize it
450 roofline_plots()
451
452
453 def arithmetic_intensity_relu():
454     n = 1024 * 1024
455     x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
456     y = torch.relu(x)
457
458     bytes = (2 * n) + (2 * n) # Read x, write y (bf16 is 2 bytes/float)
459     flops = n # n comparisons
460
461     communication_time = bytes / h100_bytes_per_sec
462     computation_time = flops / h100_flop_per_sec
463
464     Assume we can overlap communication and computation perfectly.
465     total_time = max(communication_time, computation_time)
466
467     What is the bottleneck?
468 


469 - Memory-bound: communication time > computation time

470 - Compute-bound: computation time > communication time

471 

472
473     In this case, ReLU is memory-bound.
474
475     Alternative way to see this:
476     Accelerator intensity: how much work can the accelerator do per byte transferred?
477     h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
478
479     Arithmetic intensity: how much actual work per byte for this workload?
480     arithmetic_intensity = flops / bytes # ~1/2
481
482     What is the bottleneck
483 


484 - Memory-bound: arithmetic intensity < accelerator intensity

485 - Compute-bound: arithmetic intensity > accelerator intensity

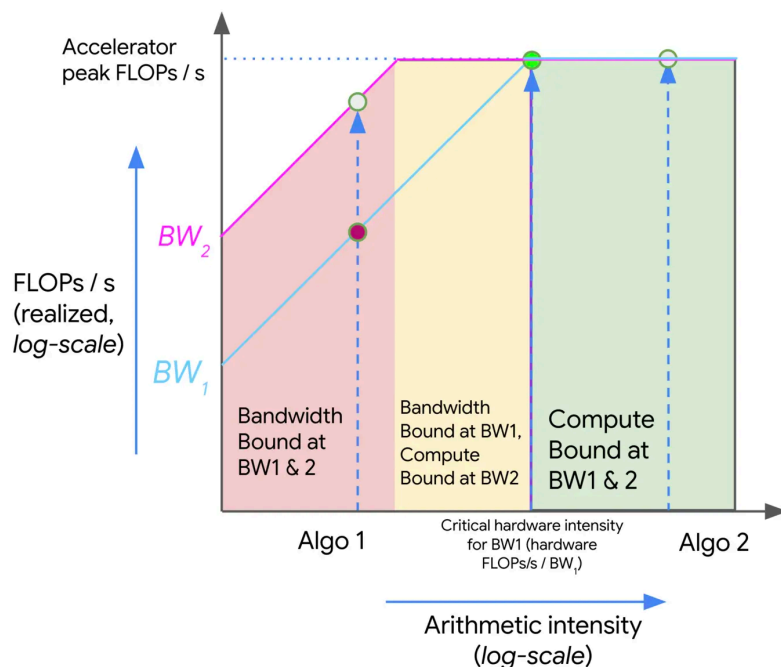
486 

487     assert arithmetic_intensity < h100_accelerator_intensity
488
489     In general, we'll find ourselves memory bound, so higher arithmetic intensity is good!
490
491
492 def arithmetic_intensity_gelu():
493     n = 1024
494     x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
495     y = F.gelu(x) # GELU(x) = 0.5 x (1 + tanh(sqrt(2/pi) (x + 0.044715 x^3)))
496
497     bytes = (2 * n) + (2 * n) # Read x, write y (bf16 is 2 bytes/float)
498     flops = 20 * n # tanh can approximated in various ways (e.g., polynomial)
499
500     arithmetic_intensity = flops / bytes
501
502     h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec

```

```
500     assert arithmetic_intensity < h100_accelerator_intensity
501
502     Note that GeLU does more work than ReLU per byte moved, so it has higher arithmetic intensity.
503     But still memory-bound!
504     In other words, ReLU is not faster than GeLU (when doing things in an isolated way).
505
506
507     def arithmetic_intensity_dot_product():
508         n = 1024
509         x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
510         w = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
511         y = x @ w
512
513         bytes = (2 * n) + (2 * n) + 2 # Read x, read w, write y
514         flops = 2 * n - 1 # n multiplications, n-1 additions
515
516         arithmetic_intensity = flops / bytes # ~1/2
517
518         h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
519         assert arithmetic_intensity < h100_accelerator_intensity
520         Memory-bound!
521
522
523     def arithmetic_intensity_matrix_vector_product():
524         n = 1024
525         x = torch.ones(n, dtype=torch.bfloat16, device=cuda_if_available())
526         w = torch.ones(n, n, dtype=torch.bfloat16, device=cuda_if_available())
527         y = x @ w
528
529         bytes = (2 * n) + (2 * n * n) + (2 * n) # Read x, read w, write y
530         flops = n * (2 * n - 1) # n dot-products
531
532         arithmetic_intensity = flops / bytes # ~1
533
534         h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
535         assert arithmetic_intensity < h100_accelerator_intensity
536         Memory-bound!
537
538     def arithmetic_intensity_matmul():
539         n = 1024
540         x = torch.ones(n, n, dtype=torch.bfloat16, device=cuda_if_available())
541         w = torch.ones(n, n, dtype=torch.bfloat16, device=cuda_if_available())
542         y = x @ w
543
544         bytes = (2 * n * n) + (2 * n * n) + (2 * n * n) # Read x, read w, write y
545         flops = n * n * (2 * n - 1) # n^2 dot products
546
547         arithmetic_intensity = flops / bytes # ~n/3
548
549         h100_accelerator_intensity = h100_flop_per_sec / h100_bytes_per_sec
550         assert arithmetic_intensity > h100_accelerator_intensity
551         Finally, compute-bound!
552
553     As long as we have large matrices, we're compute-bound (saturating the accelerator).
554     Training Transformers involves big matrix multiplications
555     Matrix-vector product is what happens during inference, which is why inference is memory-bound.
556
557     Note: arithmetic/accelerator intensity depends on the precision (bf16 versus fp32)
558
559
560     def roofline_plots():
561         We can visualize the relationship between arithmetic intensity and performance using roofline plots.
```

562



563

- Each slice on the x-axis is a particular computation (with some arithmetic intensity)
- Each piecewise linear function corresponds to a particular hardware
- Kink is the accelerator intensity (transition from memory-bound to compute-bound)

566

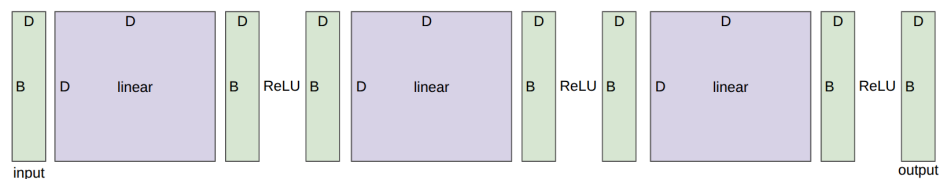
[<https://jax-ml.github.io/scaling-book/roofline/>]

568

569

```
570 def deep_linear_network():
```

571



572

Consider a deep network with L layers and D-dimensional inputs, activations, and outputs.

573

```
574 # Define the network
```

```
575 D = 8 # Dimensionality of input, activations, and output
```

```
576 L = 3 # Number of layers
```

```
577 model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())
```

578

```
579 num_parameters = get_num_parameters(model)
```

```
580 assert num_parameters == (D * D) * L
```

581

```
582 # Run the model on a batch of data
```

```
583 B = 4 # Batch size
```

```
584 x = torch.randn(B, D, device=cuda_if_available())
```

```
585 y = model(x)
```

586

587

```
588 class Block(nn.Module):
```

```
589 """Simple block that applies a linear transformation followed by a ReLU nonlinearity."""
```

```
590 def __init__(self, dim: int):
```

```
591     super().__init__()
```

```
592     self.weight = nn.Parameter(torch.randn(dim, dim) / np.sqrt(dim))
```

593

```
594 def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
595     x = x @ self.weight # Linear
```

```
596     x = F.relu(x) # Activation
```

```
597     return x
```

598

599

```
600 class DeepNetwork(nn.Module):
```

```

601     """Map `dim`-vector to a `dim`-vector."""
602     def __init__(self, dim: int, num_layers: int):
603         super().__init__()
604         self.layers = nn.ModuleList([Block(dim) for i in range(num_layers)])
605
606     def forward(self, x: torch.Tensor) -> torch.Tensor:
607         # Apply all the layers sequentially
608         for layer in self.layers:
609             x = layer(x)
610         return x
611
612
613     def optimizer():
614         Recall our deep network.
615         B = 2 # Batch size
616         D = 4 # Dimensionality of input, activations, and output
617         L = 3 # Number of layers
618         model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())
619
620         Let's define the AdaGrad optimizer
621         • momentum = SGD + exponential averaging of grad
622         • AdaGrad = SGD + averaging by grad^2
623         • RMSProp = AdaGrad + exponentially averaging of grad^2
624         • Adam = RMSProp + momentum
625
626         AdaGrad [Duchi+ 2011]
627         optimizer = AdaGrad(model.parameters(), lr=0.01)
628         state = model.state_dict()
629
630         Compute gradients
631         x = torch.randn(B, D, device=cuda_if_available())
632         y = torch.tensor([4., 5.], device=cuda_if_available())
633         pred_y = model(x).mean()
634         loss = F.mse_loss(input=pred_y, target=y)
635         loss.backward()
636
637         # Take a step
638         optimizer.step()
639         optimizer_state = {i: dict(p_state) for i, (p, p_state) in enumerate(optimizer.state.items())}
640
641         # Free up the memory
642         optimizer.zero_grad(set_to_none=True)
643

```

Memory

```

645
646     # Parameters
647     parameter_memory = 2 * (D * D * L) # (2 bytes for bf16)
648
649     # Activations
650     activation_memory = 2 * B * D * L # (2 bytes for bf16)
651
652     # Gradients
653     gradient_memory = 2 * parameter_memory # (2 bytes for bf16)
654
655     # Optimizer states
656     optimizer_state_memory = 4 * parameter_memory # (4 bytes for fp32)
657     Customary to use fp32 for stability (accumulating averages over powers over many steps)
658     4 bytes/parameter for storing second moments, Adam requires 8 bytes/parameter for storing first and second
        moments
659
660     # Putting it all together
661     total_memory = parameter_memory + activation_memory + gradient_memory + optimizer_state_memory
662

```

Compute (for one step)

```
num_parameters = D * D * L
flops = 6 * B * num_parameters
```

Transformers

The accounting for a Transformer is more complicated, but the same idea. Assignment 1 will ask you to do that.

Blog post describing memory usage for Transformer training [\[article\]](#)

Blog post describing FLOPs for a Transformer: [\[article\]](#)

```
class AdaGrad(torch.optim.Optimizer):
    def __init__(self, params: Iterable[nn.Parameter], lr: float = 0.01):
        super(AdaGrad, self).__init__(params, dict(lr=lr))

    def step(self):
        for group in self.param_groups:
            lr = group["lr"]
            for p in group["params"]:
                # Optimizer state
                state = self.state[p]
                grad = p.grad.data

                # Get squared gradients g2 = sum_{i<t} g_i^2
                g2 = state.get("g2", torch.zeros_like(grad))

                # Update optimizer state
                g2 += torch.square(grad)
                state["g2"] = g2

                # Update parameters
                p.data -= lr * grad / torch.sqrt(g2 + 1e-5)

def train_loop():
    # True linear function with weights (0, 1, 2, ..., D-1)
    D = 16 # Dimensionality
    true_w = torch.arange(D, dtype=torch.float32, device=cuda_if_available())

    # Data loader that generates (x, y) pairs
    B = 4 # Batch size
    def get_batch() -> tuple[torch.Tensor, torch.Tensor]:
        x = torch.randn(B, D).to(cuda_if_available())
        true_y = x @ true_w
        return (x, true_y)

    # Define the model and optimizer
    L = 2 # Number of layers
    model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())
    optimizer = AdaGrad(model.parameters(), lr=0.01)

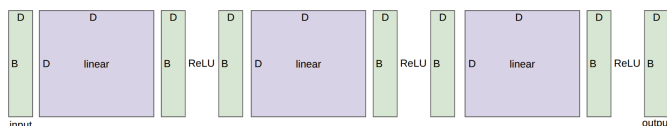
    # Train!
    num_train_steps = 10
    for t in range(num_train_steps):
        # Get data
        x, y = get_batch()

        # Forward (compute loss)
        pred_y = model(x).mean()
        loss = F.mse_loss(pred_y, y)
```

```

726         # Backward (compute gradients)
727         loss.backward()
728
729         # Update parameters
730         optimizer.step()
731         optimizer.zero_grad(set_to_none=True)
732
733
734 def gradient_accumulation():
735     Large batch sizes: improve training stability
736     However, activation memory scales with batch size, so might run out.
737     B = 1024
738     L = 16
739     D = 1024
740     activation_memory = B * L * D
741     Gradient accumulation:
742     • Compute gradient on micro batches
743     • Accumulate the gradients (don't zero it out)
744     • Every batch_size / micro_batch_size steps, update the parameters and zero out the gradients
745     micro_batch_size = 256
746     activation_memory = micro_batch_size * L * D
747
748
749 def activation_checkpointing():
750     For training, we need to store the activations of all layers
751     For inference, we don't compute gradients, so we only need to store the current layer's activations.
752
753

```



```

754 The memory usage is
755 B = 64      # Batch size
756 L = 16      # Number of layers
757 D = 1024    # Dimensionality
758
759 x = torch.randn(B, D, device=cuda_if_available())
760 activation_memory = B * D * 2 * L
761
762 model = DeepNetwork(dim=D, num_layers=L).to(cuda_if_available())
763 memory = get_max_memory_usage(lambda: model(x).backward())
764
765 Can we reduce this?
766
767 Activation checkpointing = gradient checkpointing = rematerialization
768 Key idea:
769 • Forward pass: keep only activations at subset of layers
770 • Backward pass: recompute the missing activations from the last checkpoint
771 Philosophy: tradeoff memory for compute
772
773 # Store all activations:  x g1 h1 g2 h2 g3 h3 g4 h4
774 # Activation checkpointing: x   h1   h2   h3   h4
775
776 # Define the model with checkpointing
777 model = DeepNetworkCheckpointed(dim=D, num_layers=L).to(cuda_if_available())
778 checkpointed_memory = get_max_memory_usage(lambda: model(x).backward())
779
780 Can we reduce this even more, especially for deep networks (large L)?
781
782 # Store all layers:  | h1 h2 h3 h4 h5 h6 h7 h8 h9 |
783 # Store no layers:   |                               |
784 # Store some layers: |   h3       h6       h9   |
785
786 How frequently to checkpoint?

```

- If store each layer's activations, then activation memory is $O(L)$ and no recomputation.
- If store no activations, then activation memory is $O(1)$ and compute is $O(L^2)$ (recompute from the start for each layer).
- If store every $\sqrt{L}$ layers, then activation memory is $O(\sqrt{L})$ and $O(\sqrt{L})$ recomputation, balanced.

```

792 class DeepNetworkCheckpointed(nn.Module):
793     """Same as DeepNetwork, but with activation checkpointing."""
794     def __init__(self, dim: int, num_layers: int):
795         super().__init__()
796         self.layers = nn.ModuleList([Block(dim) for i in range(num_layers)])
797
798     def forward(self, x: torch.Tensor) -> torch.Tensor:
799         # Apply all the layers sequentially
800         for layer in self.layers:
801             # KEY: line: we only store activations
802             x = torch.utils.checkpoint.checkpoint(layer, x)
803         return x
804
805     def get_max_memory_usage(func):
806         """Measure how much memory calling `func` uses."""
807         if not torch.cuda.is_available():
808             return 0 # Can't measure it without GPUs!
809
810         torch.cuda.empty_cache()
811         torch.cuda.reset_peak_memory_stats()
812         func()
813         return torch.cuda.max_memory_allocated()
814
815     #####
816
817     def get_memory_usage(x: torch.Tensor):
818         return x.numel() * x.element_size()
819
820
821
822     def get_promised_flop_per_sec(dtype: torch.dtype) -> float:
823         """Return the peak FLOP/s for `device` operating on `dtype`."""
824         if not torch.cuda.is_available():
825             # No CUDA device available, so can't get FLOP/s
826             return 1
827         properties = torch.cuda.get_device_properties(cuda_if_available())
828
829         if "A100" in properties.name:
830             # https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/a100/pdf/nvidia-a100-datasheet-us-nvidia-
1758950-r4-web.pdf
831             if dtype == torch.float32:
832                 return 19.5e12
833             if dtype in (torch.bfloat16, torch.float16):
834                 return 312e12
835             raise ValueError(f"Unknown dtype: {dtype}")
836
837         if "H100" in properties.name:
838             # https://www.nvidia.com/en-us/data-center/h100/
839             if dtype == torch.float32:
840                 return 67.5e12
841             if dtype in (torch.bfloat16, torch.float16):
842                 return 1979e12 / 2 # 1979 is for sparse, dense is half of that
843             raise ValueError(f"Unknown dtype: {dtype}")
844
845         if "B200" in properties.name:
846             # https://www.primeline-solutions.com/media/categories/server/nach-gpu/nvidia-hgx-h200/nvidia-blackwell-b200-
datasheet.pdf
847             if dtype == torch.float32:

```

```
848         return 75e12
849     if dtype in (torch.bfloat16, torch.float16):
850         return 4.5e15 / 2 # 4.5e15 is for sparse, dense is half of that
851     raise ValueError(f"Unknown dtype: {dtype}")
852
853
854 def time_matmul(a: torch.Tensor, b: torch.Tensor) -> float:
855     """Return the number of seconds required to perform `a @ b`."""
856
857     # Wait until previous CUDA threads are done
858     if torch.cuda.is_available():
859         torch.cuda.synchronize()
860
861     def run():
862         # Perform the operation
863         a @ b
864
865     # Wait until CUDA threads are done
866     if torch.cuda.is_available():
867         torch.cuda.synchronize()
868
869     # Time the operation `num_trials` times
870     num_trials = 5
871     total_time = timeit.timeit(run, number=num_trials)
872
873     return total_time / num_trials
874
875
876 def get_num_parameters(model: nn.Module) -> int:
877     return sum(param.numel() for param in model.parameters())
878
879
880 if __name__ == "__main__":
881     main()
```